

Análisis Sintáctico

Cap. 4 y 5 Louden

Prof: Ing. Julio Paciello

Análisis Sintáctico Descendente (Top Down Parsing)

- Arriba a abajo
- Derivación por la izquierda
 - *Backtracking Parser*
 - *Predictive Parser*
- Tipos de Análisis
 - Descendente Recursivo
 - $LL(1) \Rightarrow LL(k)$
 - L -> Fuente de Izq a Der
 - L -> Derivación por Izq

Descendente Recursivo

- Ej. 1:

exp -> exp addop term/term

addop -> +|-

term -> term (multop factor|factor

multop -> *

factor -> (exp) | number

1 función por cada No Terminal

```
procedure factor;  
  begin  
    case token of  
      ( : match ( ( );  
        exp;  
        match ( ) );  
        number:  
        match (number);  
      else error;  
    end case;  
end factor;
```

```
procedure match (expToken)  
  begin  
    if token == expToken then  
      getToken;  
    else  
      error;  
    end if;  
end match;
```

Descendente Recursivo (2)

- **Ej. 2:**

if-stmt -> if (exp) statement
 |if (exp) statement else statement

```
procedure if-stmt;  
  begin  
    match (if);  
    match (();  
    exp;  
    match ());  
    statement;  
    if (token = else) then  
      match (else);  
      statement;  
    end if;  
  end if-stmt;
```

EBNF

If_stmt -> if (exp) statement
 [else statement]

¿Cuál producción elegir?

$A \rightarrow \alpha \mid \beta \mid \dots$

¿ Cuándo usar α y cuándo β ?

- Conjuntos Primero de α y β : conjunto de caracteres que pueden empezar una cadena.
- Siguiente (A): conjunto de caracteres que pueden venir después que A

LL (1) Parsing

- Simulación de recursión con pila

Ej: $s \rightarrow (s) s | \epsilon$

Parsing Stack	Input	Action
\$ S	() \$	$S \rightarrow (S) S$
\$ S) S (	() \$	match
\$ S) S	) \$	$S \rightarrow \epsilon$
\$ S)	) \$	match
\$ S	\$	$S \rightarrow \epsilon$
\$	\$	Accept

Algoritmo LL(1)

1. Colocar el símbolo inicial en la pila

Se acepta la cadena si luego de aplicar derivaciones la pila queda vacía

2. Reemplazar el No Terminal A del tope por una producción del tipo $A \rightarrow \alpha$

Paso de “Generar”

1. Hacer “matching” del token del tope con el siguiente input token

Paso de “Matching”

Tabla de Parsing

1. Si $A \rightarrow \alpha$ es una producción y existe $\alpha \Rightarrow^* a \beta$ donde a es token, añadir $M[A;a] = A \rightarrow \alpha$
2. Si $A \rightarrow \alpha$ es una producción y existe $\alpha \Rightarrow^* \epsilon$ y $S\$ \Rightarrow^* \beta A a \gamma$ donde S es el simbolo inicial y a es un token o \$, agregar $M[A,a] = A \rightarrow \alpha$ (caso especial $\alpha = \epsilon$)

Explicación: Si A produce vacío mediante α , entonces el token a puede venir despues de A

Ej: $S \rightarrow (S)S \mid \epsilon$

M[N,T]	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

Algoritmo Table-based

```
while (tope != $ and token != $) {  
    if(tope es un Terminal “a” and token==“a”)  
        pop “a”;  
        token = getToken();  
    else if (tope es un No Terminal A and token==“a” and  
        existe  $T[A, a]$  que contiene  $A \rightarrow X_1 X_2 \dots X_n$ )  
        pop A;  
        push  $X_n, X_{n-1}, \dots, X_1$ ;  
    else error;  
}  
if(tope == $ y token == $) accept;  
else error;
```

Recursión por Izquierda

1. Inmediata

$A \rightarrow A\alpha \mid \beta$
no

α y β : *son T y NT que*

$A \rightarrow \beta A'$

comienzan con A

$A' \rightarrow \alpha A' \mid \varepsilon$

- General Inmediata

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \varepsilon$

Recursión por Izquierda

1. General

$$\begin{array}{l} A \rightarrow B b \mid c \\ B \rightarrow A a \mid d \end{array}$$

$$\begin{array}{l} A \rightarrow B b \mid c \\ \underline{B \rightarrow Bba \mid ca \mid d} \\ A \rightarrow B b \mid c \\ B \rightarrow caB' \mid dB' \\ B' \rightarrow ba B' \mid \varepsilon \end{array}$$

ordenar los NT $\{A_1, A_2, \dots, A_m\}$

desde $i=1$ to m

desde $j=1$ to $i-1$

reemplazar $A_i \rightarrow A_j \beta$ por

$A_i \rightarrow \alpha_1 \beta \mid \dots \mid \alpha_k \beta$ donde

$A_j \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ es la regla

actual para A_j

Factordeo por Izquierda

$A \rightarrow \alpha\beta \mid \alpha y$ Ej 1: if_stmt $\rightarrow$ if (exp) stmt
 | if (exp) stmt else stmt

$A \rightarrow \alpha A'$

$A' \rightarrow \beta \mid y$ if_stmt $\rightarrow$ if(exp) stmt opcion
 opcion $\rightarrow$ else stmt | ϵ

Ej_2: stmt_seq $\rightarrow$ stmt; stmt_seq | stmt
stmt $\rightarrow$ S

```
stmt_seq -> stmt stmt'
stmt'   -> ; stmt_seq | ε
stmt    -> S
```

Algoritmo para computar First(A)

```
for all No terminal A do First(A) = { };  
  while existan cambios a cualquier First(A) do  
    foreach production A → X1X2...Xn do  
      K=1; continue = true;  
      while continue = true and k ≤ n do  
        Add First(Xk) - {ε} to First(A)  
        if ε is not in First(Xk) then  
          Continue = false;  
        K=k+1  
      if continue = true then add ε to First(A)
```

Ejercicio (Pg. 175)

$\text{exp} \rightarrow \text{exp addop term}$

$\text{exp} \rightarrow \text{term}$

$\text{addop} \rightarrow +$

$\text{addop} \rightarrow -$

$\text{term} \rightarrow \text{term mulop factor}$

$\text{term} \rightarrow \text{factor}$

$\text{mulop} \rightarrow *$

$\text{factor} \rightarrow (\text{exp})$

$\text{factor} \rightarrow \text{number}$

$\text{First}(\text{exp}) = \{ (, \text{number} \}$

$\text{First}(\text{addop}) = \{ +, - \}$

$\text{First}(\text{term}) = \{ (, \text{number} \}$

$\text{First}(\text{mulop}) = \{ * \}$

$\text{First}(\text{factor}) = \{ (, \text{number} \}$

Algoritmo para computar $\text{Follow}(A)$

1. Si A es S_0 , entonces $\$$ está en $\text{Follow}(A)$
2. Si existe una producción $\beta \rightarrow \alpha A y$, entonces $\text{First}(y) - \epsilon$ está en $\text{Follow}(A)$
3. Si existe una producción $\beta \rightarrow \alpha A y$ tal que ϵ está en $\text{First}(y)$, entonces $\text{Follow}(A)$ contiene a $\text{Follow}(\beta)$

Ejercicio (Pg. 175)

$\text{exp} \rightarrow \text{exp addop term}$

$\text{exp} \rightarrow \text{term}$

$\text{addop} \rightarrow +$

$\text{addop} \rightarrow -$

$\text{term} \rightarrow \text{term mulop factor}$

$\text{term} \rightarrow \text{factor}$

$\text{mulop} \rightarrow *$

$\text{factor} \rightarrow (\text{exp})$

$\text{factor} \rightarrow \text{number}$

$\text{First}(\text{exp}) = \{ (, \text{number} \}$

$\text{First}(\text{addop}) = \{ +, - \}$

$\text{First}(\text{term}) = \{ (, \text{number} \}$

$\text{First}(\text{mulop}) = \{ * \}$

$\text{First}(\text{factor}) = \{ (, \text{number} \}$

$\text{Follow}(\text{exp}) = \{ \$, +, -,) \}$

$\text{Follow}(\text{addop}) = \{ (, \text{number} \}$

$\text{Follow}(\text{term}) = \{ \$, +, -,), * \}$

$\text{Follow}(\text{mulop}) = \{ (, \text{number} \}$

$\text{Follow}(\text{factor}) = \{ \$, +, -,), * \}$

Construcción de Tablas de Parsing LL(1)

Repetir p/ cada no terminal A y producción $A \rightarrow X$

1. Para cada Token a en $\text{First}(\alpha)$, agregar $A \rightarrow \alpha$ en $M[A, a]$

2. Si ϵ está en $\text{First}(\alpha)$, para cada a de $\text{Follow}(A)$ (a es token o \$), agregar $A \rightarrow \alpha$ a $M[A, a]$

Manejo de Errores

- Recuperación de Errores
 - Panic Mode
- Corrección de Errores
 - Minimal Distance Error Correction

Error Recovery

- Panic Mode:
 - Parametrizar cada procedimiento recursivo con un synchronizing set.
 - GetToken hasta alcanzar \$ encontrar un token synchronizing.
- Terminar Compilación
 - Panic Mode sin sincronización

Error Correction

- Minimal Distance Error Correction
 - Modificar el fuente para corregir el error

Características deseables:

- Eficiencia
- Evitar Error Cascade
- Evitar Loops Infinitos
- Parsear la mayor cantidad de Tokens

Ej: Panic Mode con Synch

```
procedure exp (synchset)
begin
    check input ({(, number}, synchset);
    if not (token in synchset) then
        term(synchset);
        while token = + or token = - do
            match(token);
            term(synchset);
        end while;
        checkinput (synchset, {(, number});
    end if;
end exp;
```

Ej: Panic Mode con Synch (2)

```
procedure check input(firsts, follows)
begin
    if not (token in firsts) then
        error;
        scanto (first U follows);
    end if;
end check input;
```

```
procedure scanto (synchset)
begin
    while not (token in synchset U $) do
        getToken;
    end;
end scanto;
```

```
procedure factor (synchset)
begin
    checkinput ({(, num}, synchset);
    if not (token in synchsets) then
        case token of
            (: match( ( );
                exp( { ) } );
                match( ) );
            number;
                match (number);
            else error;
        end case;
        checkinput(synchset,
                    ({(, num}));
    end if;
end factor;
```

Panic Mode con Synch - LL

- Para cada No Terminal A en la tabla de Parsing, y para cada token que no está en $\text{First}(A)$ ni en $\text{Follow}(A)$ si ϵ estuviera en $\text{First}(A)$, colocar:
 - Pop: Si el token está en $\text{Follow}(A)$ o es $\$$. Indica que se debe extraer A de la pila (equivalente a generar una producción con ϵ).
 - Scan: Si el token no está en $\text{First}(A) \cup \text{Follow}(A)$. Indica que se descarta el token del input y se avanza al siguiente token.

Análisis Ascendente

- Análisis de la cadena de izquierda a derecha
- Derivación por derecha
- Construcción de árbol sintáctico de forma ascendente

Operaciones

- **Desplazamiento (shift):** Aplicar a terminales
- **Reducción:** Sustituir una subcadena de simbolos en la pila que representen la parte derecha de una produccion por el No Terminal de la parte izquierda de la produccion (No Terminales y Terminales)

Técnicas

- LR (0) : sin pre-análisis
- SLR(1) : simple LR con un símbolo de preanálisis
- LR(1) : LR con un símbolo de preanálisis
- LALR(1): Look-Ahead LR, intermedio entre SRL(1) y LR(1) en costo y potencia.
- LR(k) : LR con k símbolos de preanálisis

Porqué LR más poderoso que LL?

- LR – k símbolos de pre-análisis, y conociendo lo que deriva por el lado derecho (símbolos en la pila)
- LL – solo k símbolos de pre-análisis.

Mango: conjunto de terminales y no terminales en la cima de la pila que forman la parte derecha de una producción y que pueden ser reducidos.

LR(0)

- Ej 1:
 $S' \rightarrow S$
 $S \rightarrow (S)S \mid E$
Entrada: $()\$$



Gramática aumentada

$S' \rightarrow S$
 $S \rightarrow \dots$
Siendo

S el S original

Pila	Input	Acción
\$	$()\$$	Despl.
$\$($	$)\$$	Reducir $S \rightarrow E$
$\$(S$	$)\$$	Despl.
$\$(S)$	\$	Reducir $S \rightarrow E$
$\$(S)S$	\$	Reducir $S \rightarrow (S) S$
$\$ S$	\$	Reducir $S' \rightarrow S$
$\$ S'$	\$	accept

Inicio: con la Pila vacía

Aceptación: con término inicial en el tope de la pila

- Ej 2:

$E' \rightarrow E$

$E \rightarrow E+n$

$E \rightarrow n$

Pila	Input	Acción
\$	n+n \$	Despl.
\$ n	+n \$	Reducir $E \rightarrow n$
\$E	+n \$	Despl.
\$E+	n \$	Despl.
\$E+n	\$	Reducir $E \rightarrow E+n$
\$ E	\$	Reducir $E' \rightarrow E$
\$ E'	\$	Accept

Problema del LR (0)

Conflicto desp/reducir

$E' \rightarrow E$

y $E \rightarrow E+n$ porque no
hace pre-análisis



SLR(1)

- Utiliza el símbolo de pre-análisis para determinar si reducir o desplazar.
- Si el pre-análisis esta en sgte (elem. de la cima), entonces reducir.

*YACC. Generador de Analizadores
Sintácticos LR*

Ejercicios

Lisas de Prolog

$L' \rightarrow \text{Lista}$

$\text{Lista} \rightarrow []$

$\quad | [\text{Term}]$

$\text{Term} \rightarrow \text{Term}, \text{Term}$

$\quad | \text{Lista} \mid \text{id}$

Pila	Input	Acción
\$	[id,id]\$	Despl.
\$[	id,id]\$	Despl.
\$[id	,id]\$	Red. $\text{Term} \rightarrow \text{id}$
\$[term	,id]\$	Despl.
\$[term,	id]\$	Despl.
\$term,id	]\$	Red. $\text{Term} \rightarrow \text{id}$
\$[term,term	]\$	Despl.
\$[term,term	]\$	Red. $\text{Term}, \text{Term} \rightarrow \text{Term}$
\$[term	]\$	Despl.
\$[term]	\$	Red. $\text{Lista} \rightarrow [\text{Term}]$
\$ Lista	\$	Red. $L' \rightarrow \text{Lista}$
\$ L'	\$	aceptar

Ejercicios

1. $S' \rightarrow \text{stmt-seq}$
 $\text{stmt-seq} \rightarrow \text{stmt-seq}; \text{stmt} \mid \text{stmt}$
 $\text{stmt} \rightarrow s$
Entrada: s; s; s

Ejercicios

Pila	Input	Acción
\$	s; s; s \$	Despl.
\$ s	; s; s \$	Red. stmt -> s
\$ stmt	; s; s \$	Red. stmt -> stmt-seq
\$ stmt-seq	; s; s \$	Despl.
\$ stmt-seq;	s; s \$	Despl.
\$ stmt-seq; s	; s \$	Red. stmt -> s
\$ stmt-seq; stmt	; s \$	Red. stmt-seq -> stmt-seq; stmt
\$ stmt-seq	; s \$	Despl.
\$ stmt-seq;	s \$	Despl.
\$ stmt-seq; s	\$	Red. stmt -> s
\$ stmt-seq; stmt	\$	Red. stmt-seq -> stmt-seq; stmt
\$ stmt-seq	\$	Despl.
\$ S'	\$	Despl.

Consultas?